多设备 AI 自动化控制网络

备注：windows使用 PowerShell 运行

PowerShell Installer.exe
1.07MB

OpenClaw + 飞书 + Windows 自动化控制系统部署文档

1.1 项目目标

构建一个 **通过飞书聊天控制本地电脑的 AI 自动化系统**。

原始情况下，OpenClaw 部署在某一台宿主机上时，AI Agent 运行在其受限的沙盒环境中，默认只能访问本机的工作区和系统资源，因此所有自动化操作基本局限于 **宿主机本身**，无法直接控制或操作其他电脑的文件系统、应用程序或桌面环境。通过引入 **OpenClaw Node + Tailscale 网络 + Windows 自动化桥接（desktopctl）** 的方案，我们将执行能力从单一宿主机扩展到多个设备：Agent 仍然在沙盒中运行，但通过 Node 协议将任务转发到不同的远程节点执行，从而实现 **跨沙盒、跨设备的控制能**

力。这样，AI 不再只控制部署 OpenClaw 的那台机器，而是可以通过统一的 Gateway 调度多台 Windows、Linux 或其他设备，形成一个可扩展的 **多设备 AI 自动化控制网络**。

> 备注：所有电脑不需要在一个局域网下

典型使用场景：

代码块

```
飞书: 帮我读取 C:\Github 目录
ubuntu OpenClaw: 调用 Windows Node
Windows: 返回文件列表
```

---

2. 系统架构

2.1 总体架构

代码块

```
                    ┌──────────────────┐
                    │     Feishu       │
                    │    飞书机器人    │
                    └──────────┬───────┘
                               │
                               │ Webhook
                               ▼
                 ┌──────────────────────────┐
                 │   OpenClaw Gateway       │
                 │      Ubuntu              │
                 │                          │
                 │   AI Agent              │
                 │   Model Router          │
                 └──────────┬──────────────┘
                            │
                            │ Tailscale VPN
                            ▼
                   ┌──────────────────┐
                   │  Windows Node    │
                   │   WinDesk        │
                   │                  │
                   │ desktopctl.py    │
                   │                  │
                   │ 自动化控制层     │
                   └──────────────────┘
```

2.2 模型架构

当前 AI 模型配置：

```
代码块

Primary Model
    ↓
Gemini 3.1 Pro

Fallback Model
    ↓
Kimi K2.5
```

模型路由逻辑：

```
代码块

请求
 ↓
Gemini
 ↓
失败 / 限流
 ↓
Kimi
```

3. 系统组件

组件	作用
Feishu Bot	用户入口
OpenClaw Gateway	AI 调度中心
AI Agent	任务执行逻辑
Tailscale	私有网络
Windows Node	被控制电脑
desktopctl	Windows 自动化控制

4. 部署流程

5.1 Ubuntu 安装 OpenClaw

> https://docs.openclaw.ai/pi
>
> https://docs.openclaw.ai/install#from-source

安装 openclaw：

```
代码块
curl -fsSL https://openclaw.ai/install.sh | bash
```

启动 Gateway：

```
代码块
openclaw gateway run
```

默认端口：

```
代码块
127.0.0.1:18789
http://127.0.0.1:18789
```

5.2 配置 Gateway

配置文件：

```
代码块
~/.openclaw/openclaw.json
```

示例配置：

openclaw.json

3.77KB

代码块

```
{
  "gateway": {
    "port": 18789,
    "bind": "loopback",
    "auth": {
      "mode": "token"
    }
  }
}
```

5.3 飞书机器人接入

创建飞书应用：

代码块

```
飞书开放平台
→ 创建机器人
→ 获取 webhook
```

配置 OpenClaw Feishu 插件。

插件路径：

代码块

```
~/.openclaw/extensions/feishu
```

消息流程：

代码块

```
飞书
 ↓
Webhook
 ↓
```

```
OpenClaw Agent
 ↓
执行任务
```

5.4 Windows 安装 Node

Windows 安装安装 OpenClaw：

代码块

```
iwr -useb https://openclaw.ai/install.ps1 | iex
```

5.5 启动 Windows Node

代码块

```
宿主机的token在~/.openclaw/openclaw.json中找到
```

运行：

代码块

```
windows:
setx OPENCLAW_GATEWAY_TOKEN "宿主机openclawtoken"

openclaw node run \
--host rocos-thinkstation-p3-tower.tailce3797.ts.net \
--port 443 \
--tls \
--display-name WinDesk
ubuntu:
配对:
openclaw devices list（找到准备配对的ID）
openclaw devices approve <requestId>


```

参数说明：

参数	说明
host	Gateway 地址
port	Gateway 端口
tls	使用安全连接
display-name	节点名称

```
◇
Paired (4)
```

Device	Roles	Scopes	Tokens	IP
c0244f9ad6bfd8e42e48bd3c3dada8b971130adc0bac7764d99a8e17a0d98dad	operator	operator.admin, operator.approvals, operator.pairing	operator	
a1f7143fa882558d30b6fdaf5db7606d346e6b69c319236f0e858f1a7d01de75 WinDesk	operator, node	operator.read, operator.admin, operator.write, operator.approvals, operator.pairing	node, operator	
c6e2afb108fb34d9cc78e500e0ea6f4a64bb70a0834a0c87974984476970 7937	node operator	operator.admin, operator.approvals, operator.pairing	node operator	

5.6 Node 配对检查

查看节点：

代码块

```
1   openclaw nodes status
```

示例输出：

```
Known: 2 · Paired: 2 · Connected: 1
```

Node	ID	IP	Detail	Status	Caps
rocos-ThinkStation-P3-Tower	a1f7143fa882558d30b6fdaf5db7606d346e6b69c319236f0e858f1a7d01de75		core v2026.3.7 · path: /usr/local/bin:~/.local/bin:…:~/.local/share/pnpm	paired · connected (3h ago)	browser, system
WinDesk	926019f6db85a6caf6f7f6a0d3d2dc91b40515e7fb30213214b805a6ad02d96a			paired · disconnected	

代码块

```
Known: 2
Paired: 2
Connected: 2
```

节点：

代码块

```
Ubuntu Gateway
Windows WinDesk
```

---

5. 网络架构

使用 **Tailscale VPN** 连接设备。

tailscale-setup-1.94.2.exe
1.33MB

> linux：curl -fsSL https://tailscale.com/install.sh | sh

本方案依赖 **Tailscale VPN 内网**。

当前设备：

设备	角色	Tailscale IP
Ubuntu	OpenClaw Gateway	100.126.182.50
Windows	OpenClaw Node	100.64.46.114

访问方式：

Windows Node

│

│ ws / https

▼

Ubuntu Gateway

127.0.0.1:18789

Tailscale 将其暴露为：

https://rocos-thinkstation-p3-tower.tailce3797.ts.net

5.1 Ubuntu 配置 Gateway

配置文件：

~/.openclaw/openclaw.json

Gateway 配置：

代码块

```
"gateway": {
  "port": 18789,
  "mode": "local",
  "bind": "loopback",

  "controlUi": {
    "allowedOrigins": [
      "https://rocos-thinkstation-p3-tower.tailce3797.ts.net"
    ]
  },

  "auth": {
    "mode": "token",
    "token": "**********************************************"
  },

  "tailscale": {
    "mode": "serve",
    "resetOnExit": false
  }
}
```

说明：

参数	作用
port	Gateway 端口
bind=loopback	只监听本机
allowedOrigins	允许远程 Control UI
auth.token	Gateway认证
tailscale.serve	自动用 Tailscale 暴露服务

5.2 重启 Gateway

修改配置后必须重启：

```
openclaw gateway restart
```

或

```
systemctl --user restart openclaw-gateway
```

5.3 Tailscale Serve

查看服务暴露情况：

```
tailscale serve status
```

输出：

> https://rocos-thinkstation-p3-tower.tailce3797.ts.net (tailnet only)
>
> |-- / proxy http://127.0.0.1:18789

含义：

```
代码块
公网入口
↓
ts.net 域名
↓
Tailscale 内网
↓
127.0.0.1:18789
```

5.4 检查 Ubuntu Tailscale IP

运行：

```
代码块
tailscale ip -4
```

结果：

```
代码块
100.126.182.50
```

5.5Windows Node

Windows 机器：

```
代码块
Tailscale IP
100.64.46.114
```

Node 通过：

```
代码块
wss://rocos-thinkstation-p3-tower.tailce3797.ts.net
```

连接 Gateway。

5.6 网络连通性验证

在 Windows 测试：

ping Gateway

```
代码块
1   ping 100.126.182.50
```

ping ts 域名

```
代码块
1   ping rocos-thinkstation-p3-tower.tailce3797.ts.net
```

Tailscale 域名：

```
代码块
1   rocos-thinkstation-p3-tower.tailce3797.ts.net
```

6. Windows 自动化控制

> 备注：
>
> 方案本质上是控制 **Windows 当前桌面会话。Windows 处于已登录、未锁屏、桌面活跃的状态**
>
> ### A. 值得开发
>
> - 读写指定目录文件
> - 搜索、整理、重命名、归档文件
> - 打开 VSCode、Chrome、资源管理器、Office
> - 浏览器自动化登录内部系统、填表、下载报表
> - 截图、保存证据、上传结果

- 自动运行 Python / PowerShell / bat
- 自动处理 Git 仓库
- UI 自动化企业桌面软件
- 飞书一句话触发完整工作流

B. 更多工程量

- OCR 识别复杂界面
- 智能点击按钮
- 自动处理不规则弹窗
- 多应用协同
- 跨窗口流程编排
- 自主恢复失败步骤
- 长流程任务监控与重试

创建控制脚本：

代码块

```
desktopctl.py

pip install flask pyautogui mss pyperclip(系统python最好)
```

功能：

代码块

```
list-files
read-file
open-app
screenshot
mouse-click
type-text
hotkey
```

创建 bat 启动：

代码块

```
desktopctl.bat
```

源码：

desktopctl.bat
42 B

desktopctl.py
4.46KB

技能：

windesk-desktopctl.zip
5.26KB

---

7. OpenClaw 执行权限

ubuntu添加 allowlist：

代码块

```
openclaw approvals allowlist add \
--node WinDesk \
"C:\openclaw\desktopctl.bat"

openclaw approvals allowlist add --node WinDesk "C:\Windows\System32\cmd.exe"
```

测试功能正常的代码

代码块

```
openclaw nodes run --node WinDesk -- "C:\Windows\System32\cmd.exe" /c "C:\openclaw\desktopctl.bat list-files --path C:\Github"

```

---

8. 模型配置

> https://zhuanlan.zhihu.com/p/2013328075006444391

---

9. 常见问题

9.1 Node 连接失败

错误：

```
代码块
SECURITY ERROR
Cannot connect over ws
```

原因：

OpenClaw 需要 TLS。

解决：

```
代码块
--tls
```

9.2 Clash 导致连接失败

错误：

```
代码块
ERR_CONNECTION_ABORTED
ERR_CONNECTION_CLOSED
```

原因：

Clash TUN 拦截 Tailscale。

解决：

退出代理，直连即可

9.3 origin not allowed

错误：

```
origin not allowed
```

原因：

Control UI 只允许 localhost。

解决：

配置：

```
allowedOrigins
```

9.4 Python 模块缺失

错误：

```
ModuleNotFoundError: pyautogui
```

解决：

```
pip install pyautogui pillow opencv-python
```

9.5 API rate limit

错误：

```
Provider google is in cooldown
```

原因：

Gemini API 限流。

解决：

使用 fallback：

```
代码块
1   Kimi
```

---

10. 系统能力

系统现在支持：

```
代码块
1   飞书聊天
2    ↓
3   AI Agent
4    ↓
5   控制 Windows
```

能力：

- 文件读取
- 应用启动
- 屏幕截图
- 鼠标控制
- 键盘输入

---

11. 部署 Checklist

部署前检查：

Gateway

- Ubuntu 安装 Node.js
- 安装 OpenClaw

- Gateway 启动

网络

- Tailscale 安装
- Ubuntu 加入网络
- Windows 加入网络

Node

- Windows 安装 OpenClaw
- Node 启动
- Node connected

自动化

- desktopctl.py 创建
- Python 依赖安装

权限

- allowlist 添加

飞书

- 飞书机器人创建
- webhook 配置

---

12. 最终系统结构

代码块

```
Feishu
   │
   ▼
OpenClaw Gateway (Ubuntu)
   │
   ├── AI Models
   │      ├─ Gemini
   │      └─ Kimi
   │
   ├── Agents
   │
```

```
    └── Nodes
          │
          └── Windows
                │
                └── Desktop Automation
```